

RAPPORT DE STAGE

Projet SuperCar

Documentation technique

Site web dynamique et espace d'administration sécurisé

RÉALISÉ PAR

Niela

Ilaina

FORMATION

BTS SIO – option SLAM

MCCI Business School

Julien

Sommaire

I – Contexte du projet 3

1. Présentation de l'organisation cliente 3
2. Cahier des charges et prestations attendues 3
3. Objectifs du projet 3
4. Problématique 3
5. Démarche du projet 4

II – Conception et planification 4

6. Analyse des besoins et choix techniques 4
7. Étapes de conception 4
8. Répartition des rôles 5
9. Organisation du travail 5

III – Réalisation technique 6

10. Architecture globale (MVC sécurisé) 6
11. Base de données (tables et relations) 8
12. Description des pages de l'administration 11
13. Fonctionnalités spécifiques 15
14. Charte graphique et identité visuelle (thème blanc/orange) 16
15. Difficultés rencontrées et solutions apportées 17

IV – Résultats obtenus 18

16. Présentation de l'administration finale 18
17. Tests et validation 18
18. Analyse de la conformité aux besoins exprimés 18

Conclusion 18

Bilan technique et personnel 18
Apprentissages et compétences développées 19
Perspectives et améliorations futures 19

I – Contexte du projet

1. Présentation de l'organisation cliente

1.1 Historique et activité

SuperCar est une concession automobile spécialisée dans la vente de voitures neuves depuis 2009. L'entreprise importe ses véhicules de plusieurs pays et dispose de quatre entrepôts ainsi que d'un siège social regroupant ses différents services.

1.2 Besoins exprimés

SuperCar souhaite améliorer sa visibilité grâce à un site web dynamique permettant de présenter ses véhicules et ses services, de faciliter les échanges avec les clients et de gérer les demandes d'essai et les messages de contact.

2. Cahier des charges et prestations attendues

Le cahier des charges prévoit la réalisation d'un site dynamique utilisant **HTML5, CSS3, PHP et MySQL**, avec une interface responsive et une partie administration sécurisée.

Le site doit notamment proposer une présentation des voitures, des services, un formulaire de contact et un système de demande d'essai. L'administration doit permettre de gérer les contenus et les données enregistrées.

3. Objectifs du projet

L'objectif principal est de développer une application web permettant à SuperCar de présenter ses véhicules et de faciliter la communication avec ses clients.

Le projet doit également permettre de centraliser les informations dans une base de données et de fournir aux administrateurs une interface permettant de gérer les voitures, les demandes et les messages.

4. Problématique

> **Comment développer une application web permettant à SuperCar de présenter ses véhicules, de faciliter les échanges avec ses clients et de gérer efficacement les demandes depuis une administration sécurisée ?**

5. Démarche du projet

Nous avons commencé par analyser le cahier des charges afin d'identifier les besoins et les fonctionnalités nécessaires. Nous avons ensuite conçu la base de données, défini les différentes pages et réparti les tâches entre les membres de l'équipe.

Le développement a ensuite été réalisé progressivement, suivi de phases de tests, de corrections et d'améliorations.

II – Conception et planification

6. Analyse des besoins et choix techniques

L'analyse des besoins a conduit au choix de plusieurs technologies : **HTML5, CSS3, Bootstrap, PHP, MySQL, JavaScript et PDO**.

L'architecture **MVC** a été utilisée afin de séparer les données, la logique de traitement et l'affichage. Les requêtes préparées avec PDO permettent également de sécuriser les échanges avec la base de données.

7. Étapes de conception

7.1 Étude du cahier des charges

Nous avons étudié les fonctionnalités demandées afin de définir l'organisation générale du site, ses pages et les données nécessaires à son fonctionnement.

7.2 Définition des pages et fonctionnalités de l'administration

L'administration comprend principalement la gestion des **voitures, clients, demandes d'essai et messages de contact**.

Elle possède également un système d'authentification permettant de protéger l'accès aux pages d'administration.

7.3 Modélisation conceptuelle et logique de la base de données

La base de données permet de centraliser les informations utilisées par l'application. Elle contient notamment les données concernant les **voitures, clients, demandes d'essai et messages de contact**.

Les données saisies par les visiteurs peuvent ainsi être retrouvées et gérées depuis l'administration.

8. Répartition des rôles

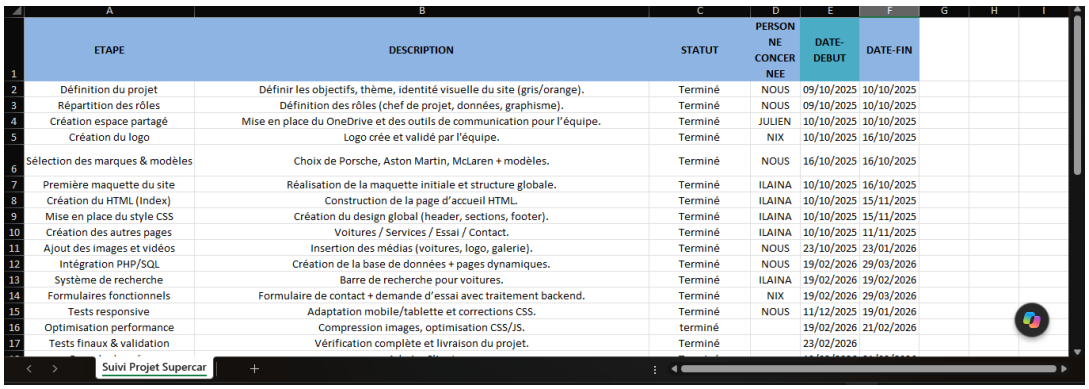
Le travail a été réparti entre les trois membres du groupe :

- **Niela** : architecture MVC, connexion, sécurité, optimisation et style ;
- **Ilaina** : administration générale, dashboard et base administration, voitures côté client, contact et demande d'essai ;
- **Julien** : services côté client, voitures et services côté administration.

Cette répartition permet à chaque membre de travailler sur des fonctionnalités précises tout en conservant une organisation commune du projet.

9. Organisation du travail

Le travail a été réparti entre les trois membres de l'équipe (Niela, Julien et Ilaina), chacun prenant en charge une partie du développement. Le suivi d'avancement a été assuré via un fichier Excel partagé, listant les tâches, la personne responsable et un statut mis à jour au fil du projet (marqué « terminé » une fois la tâche achevée). le projet a été mis en place sur GitHub afin d'assurer le versioning et de simplifier l'intégration des futures fonctionnalités."



ETAPE	DESCRIPTION	STATUT	PERSONNE CONCERNEE	DATE-DEBUT	DATE-FIN
Définition du projet	Définir les objectifs, thème, identité visuelle du site (gris/orange).	Terminé	NOUS	09/10/2025	10/10/2025
Répartition des rôles	Définition des rôles (chef de projet, données, graphisme).	Terminé	NOUS	09/10/2025	10/10/2025
Création espace partagé	Mise en place du OneDrive et des outils de communication pour l'équipe.	Terminé	JULIEN	10/10/2025	10/10/2025
Création du logo	Logo crée et validé par l'équipe.	Terminé	NIX	10/10/2025	16/10/2025
Sélection des marques & modèles	Choix de Porsche, Aston Martin, McLaren + modèles.	Terminé	NOUS	16/10/2025	16/10/2025
Première maquette du site	Réalisation de la maquette initiale et structure globale.	Terminé	ILAINA	10/10/2025	16/10/2025
Création du HTML (Index)	Construction de la page d'accueil HTML.	Terminé	ILAINA	10/10/2025	15/11/2025
Mise en place du style CSS	Création du design global (header, sections, footer).	Terminé	ILAINA	10/10/2025	15/11/2025
Création des autres pages	Voitures / Services / Essai / Contact.	Terminé	ILAINA	10/10/2025	11/11/2025
Ajout des images et vidéos	Insertion des médias (voitures, logo, galerie).	Terminé	NOUS	23/10/2025	23/01/2026
Intégration PHP/SQL	Création de la base de données + pages dynamiques.	Terminé	NOUS	19/02/2026	29/03/2026
Système de recherche	Barre de recherche pour voitures.	Terminé	ILAINA	19/02/2026	19/02/2026
Formulaires fonctionnels	Formulaire de contact + demande d'essai avec traitement backend.	Terminé	NIX	19/02/2026	29/03/2026
Tests responsive	Adaptation mobile/tablette et corrections CSS.	Terminé	NOUS	11/12/2025	19/01/2026
Optimisation performance	Compression images, optimisation CSS/JS.	terminé		19/02/2026	21/02/2026
Tests finaux & validation	Vérification complète et livraison du projet.	Terminé		23/02/2026	

Figure 1 *– Suivi des tâches dans le fichier Excel partagé*

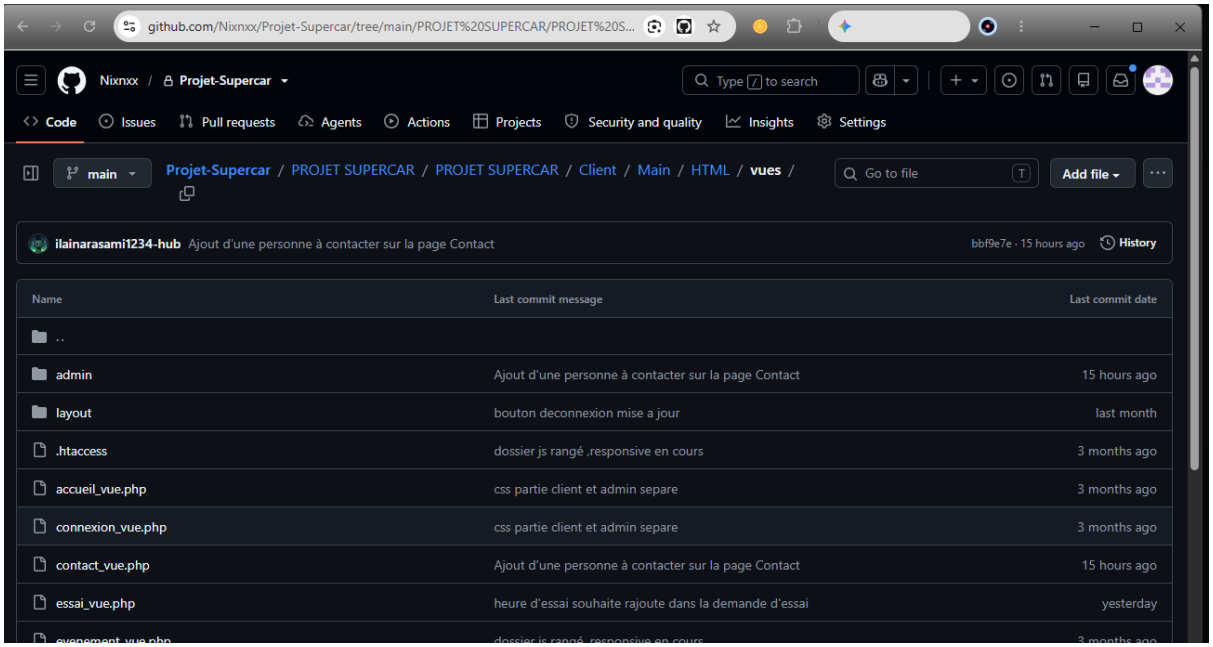


Figure 2 *– projet mis en place sur GitHub*

III – Réalisation technique

10. Architecture globale (MVC sécurisé)

Le site respecte une architecture MVC (Modèle-Vue-Contrôleur), avec une séparation nette entre l'espace client et l'espace back-office (administration) à chaque niveau :

- `config/` – paramètres de connexion et de configuration (`config.php`, `database.php`)
- `controleurs/` – logique métier, dédoublés en un dossier racine (client) et un sous-dossier `admin/`
- `modeles/` – accès aux données (requêtes SQL), également dédoublés client / `admin/`
- `vues/` – affichage (templates PHP), également dédoublés client / `admin/`
- `public/` – points d'entrée accessibles (pages PHP, CSS, JS, images), avec un sous-dossier `ADMIN/` dédié au back-office
- `includes/` – fichiers transverses de sécurité : `garde_admin.php` (contrôle d'accès), `init.php`, `securite.php`, `session.php`

Arborescence du projet (extrait)

```
├── config
│   ├── config.php
│   └── database.php
├── controleurs
│   ├── admin
│   │   ├── accueil_controleur.php
│   │   ├── accueil_mod_controleur.php
│   │   …
├── includes
│   ├── garde_admin.php
│   ├── init.php
│   ├── securite.php
│   └── session.php
├── modeles
│   ├── admin
│   │   ├── accueil_modele.php
│   │   ├── auth_modele.php
│   │   ├── client_modele.php
│   │   …
├── public
│   ├── ADMIN
│   │   ├── accueil_mod.php
│   │   ├── accueil_supp.php
│   │   ├── accueil.php
│   │   ├── admin_contact.php
│   │   ├── admin_dashboard.php
│   │   ├── admin_essais.php
│   │   ├── admin_services.php
│   │   ├── admin_voitures.php
│   │   ├── logina.php
│   │   └── logout.php
│   ├── css
│   │   ├── admin
│   │   │   ├── accueil-form.css
│   │   │   ├── accueil.css
│   │   │   ├── contact.css
│   │   │   ├── essais.css
│   │   │   ├── services.css
│   │   │   …
│   ├── Images-Index
│   │   ├── Service_Image
│   ├── js
│   │   ├── admin
│   │   │   ├── accueil-liste.js
│   │   │   ├── accueil.js
│   │   │   ├── contact.js
│   │   │   ├── essais.js
│   │   │   ├── services.js
│   │   │   └── voitures.js
│   │   ├── essai.js
│   │   ├── layout.js
│   │   ├── transparent.js
│   │   └── voitures.js
│   ├── connexion.php
│   ├── contact.php
```

```
│   ├── essai.php
│   ├── evenement.php
│   └── …
└── vues
    ├── admin
    │   ├── accueil_form_vue.php
    │   ├── accueil_list_vue.php
    │   …
    │   ├── services_vue.php
    │   └── voitures_vue.php
    ├── layout
    │   ├── footer.php
    │   ├── header.php
    │   ├── mentions_contenu.php
    │   └── politique_contenu.php
    ├── accueil_vue.php
    ├── connexion_vue.php
    ├── contact_vue.php
    ├── essai_vue.php
    ├── evenement_vue.php
    ├── inscription_vue.php
    ├── mentions_vue.php
    ├── nonconnecte_vue.php
    ├── politique_vue.php
    ├── services_vue.php
    └── voitures_vue.php
```

Mesures de sécurité

- **Requêtes préparées (MySQLi)** pour toutes les interactions avec la base de données, empêchant les injections SQL
- **Gestion des sessions PHP sécurisée** (`session.php`) pour maintenir l'état de connexion
- **Hachage des mots de passe (bcrypt)** – aucun mot de passe stocké en clair dans la table `login`
- **Contrôle d'accès admin** via `garde_admin.php`, qui vérifie le niveau de l'utilisateur (champ `niveau` de la table `login`) avant d'autoriser l'accès aux pages du dossier `ADMIN/`
- **Protection contre les attaques XSS** par échappement des données affichées
- **Protection CSRF** sur les formulaires sensibles (connexion, formulaires d'administration)
- **Upload d'images sécurisé** (contrôle du type et de l'extension des fichiers envoyés)
- **Protection contre le brute-force** : la table `login` enregistre le nombre de tentatives échouées (`tentatives_echouees`) et la date de la dernière tentative (`derniere_tentative`)

- **Traçabilité** : la table `audit` journalise les actions sensibles – par exemple, le trigger `insert_voiture` enregistre automatiquement chaque ajout de voiture (table, action, date, identifiant concerné)

11. Base de données (tables et relations)

La base de données `supercar` comprend 16 tables (moteur InnoDB, encodage utf8mb4), réparties en deux groupes :

- **Gestion opérationnelle** : `voiture`, `voiture_images`, `marque`, `client`, `login`, `demandeessai`, `contact`, `demande_service`, `service`, `proposer`, `audit`
- **Contenu de la page d'accueil (CMS)** : `accueil_bienvenue`, `accueil_bienvenue_images`, `accueil_carousel`, `accueil_points`, `accueil_stats`, `contact_config`

Principales relations entre les tables :

- `voiture` → `marque` (une voiture appartient à une marque)
- `voiture_images` → `voiture` (plusieurs images par voiture)
- `demandeessai` → `client` et `voiture` (une demande d'essai concerne un client et une voiture)
- `contact` → `client` (un message peut être lié à un client)
- `demande_service` → `client` et `service`
- `proposer` → relation N-N entre `voiture` et `service`
- `login` → `client` (compte de connexion associé à un client ; les comptes admin n'ont pas de client associé)
- `audit` → `login` et `voiture` (journalisation des actions administrateur)

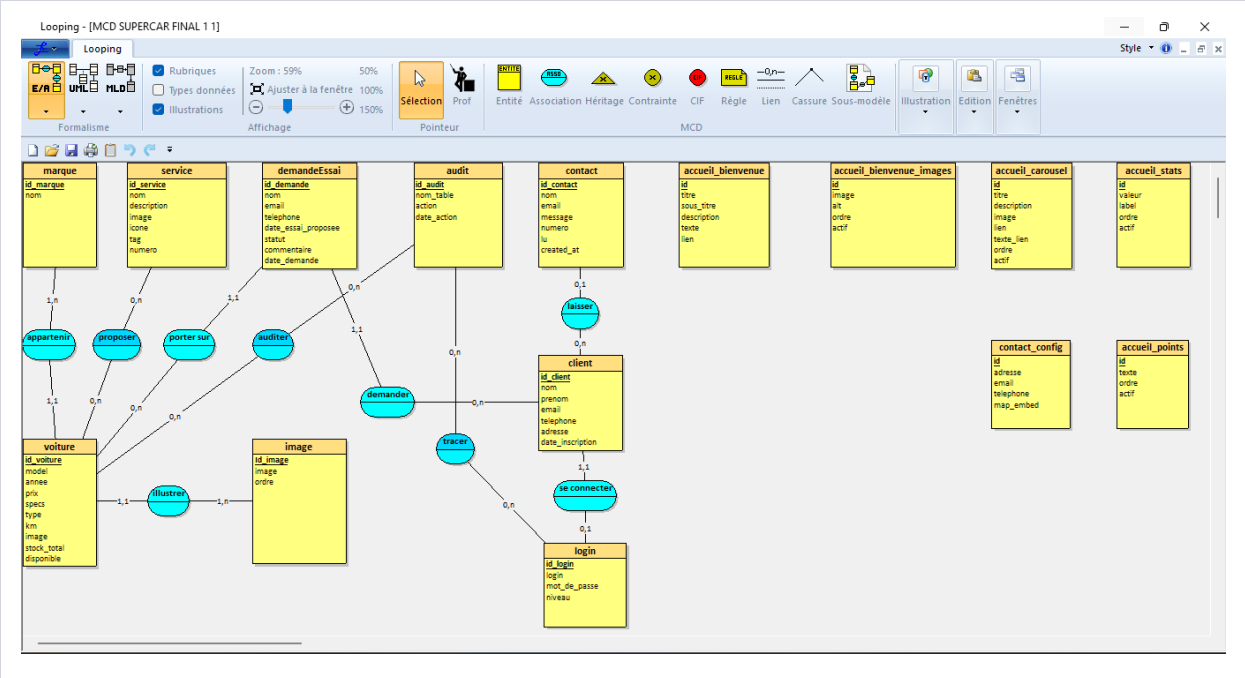


Figure 3 *– Modèle conceptuel de données de la base supercar*

11.1 Tables principales (voitures, clients, demandes d'essai, contacts)

Table voiture

Champ	Type	Description
id_voiture	int, PK, AUTO_INCREMENT	Identifiant unique de la voiture
model	varchar(50)	Modèle de la voiture
annee	date	Année du modèle
prix	bigint	Prix (Rs)
specs	varchar(300)	Caractéristiques générales
type_carburant	varchar(50)	Essence / Diesel / Électrique / Hybride
km	int	Kilométrage
image	varchar(255)	Image principale du véhicule
stock_total	int	Quantité en stock
disponible	tinyint(1)	Disponibilité (0 = non, 1 = oui)
id_marque	int, FK → marque	Marque du véhicule
transmission	varchar(20)	Automatique / Manuelle

Table client

Champ	Type	Description
id_client	int, PK, AUTO_INCREMENT	Identifiant unique du client
nom	varchar(50)	Nom
prenom	varchar(50)	Prénom
email	varchar(100)	Adresse e-mail
telephone	varchar(20)	Numéro de téléphone
adresse	varchar(100)	Adresse postale
date_inscription	date	Date de création du compte

Table demandeessai (demande d'essai)

Champ	Type	Description
id_demande	int, PK, AUTO_INCREMENT	Identifiant unique de la demande
date_demande	datetime	Date de la demande
date_essai_proposee	date	Date d'essai proposée
statut	int	Statut de la demande (en attente / validée / refusée…)
commentaire	varchar(200)	Commentaire éventuel
id_client	int, FK → client	Client à l'origine de la demande
id_voiture	int, FK → voiture	Voiture concernée

Table contact

Champ	Type	Description
id_contact	int, PK, AUTO_INCREMENT	Identifiant unique du message
nom	varchar(50)	Nom de l'expéditeur
email	varchar(100)	Adresse e-mail
message	varchar(800)	Contenu du message
numero	varchar(20)	Numéro de téléphone
id_client	int, FK → client (nullable)	Client associé, si connecté
lu	tinyint	Message lu (1) ou non lu (0)
created_at	datetime	Date d'envoi

12. Description des pages de l'administration

Le back-office (dossier `public/ADMIN/`) regroupe les pages suivantes :

- `accueil.php` / `accueil_mod.php` / `accueil_supp.php` – gestion du contenu de la page d'accueil (bloc de bienvenue, carousel, points forts, statistiques)
- `admin_dashboard.php` – tableau de bord de l'administration
- `admin_voitures.php` – gestion des voitures (voir 12.1)
- `admin_essais.php` – gestion des demandes d'essai
- `admin_contact.php` – gestion des messages de contact reçus
- `admin_services.php` – gestion des services proposés
- `logina.php` / `logout.php` – connexion et déconnexion de l'administrateur, protégées par `garde_admin.php`

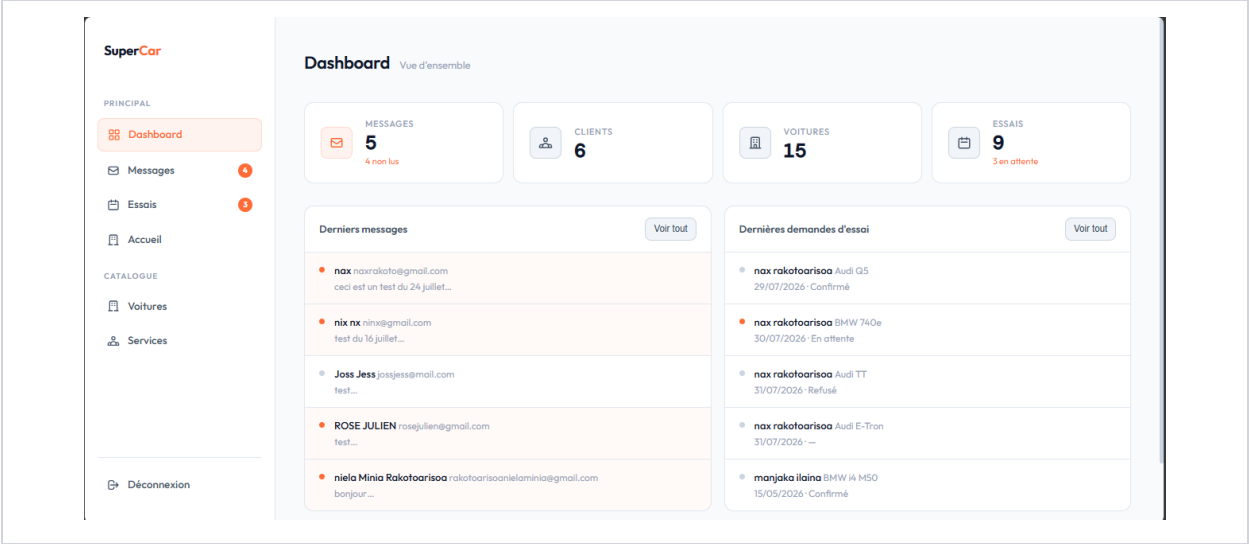


Figure 4 *– Tableau de bord de l'administration*

12.1 Gestion des voitures (CRUD)

La page `admin_voitures.php` permet une gestion complète du catalogue :

- **Create** – ajout d'une voiture (modèle, année, prix, caractéristiques, type de carburant, kilométrage, transmission, marque, stock, disponibilité) avec upload d'une ou plusieurs images (table `voiture_images`)
- **Read** – affichage de la liste des voitures du catalogue
- **Update** – modification des informations et des images d'une voiture
- **Delete** – suppression d'une voiture du catalogue
- Chaque ajout déclenche le trigger `insert_voiture`, qui journalise automatiquement l'action dans la table `audit` à des fins de traçabilité

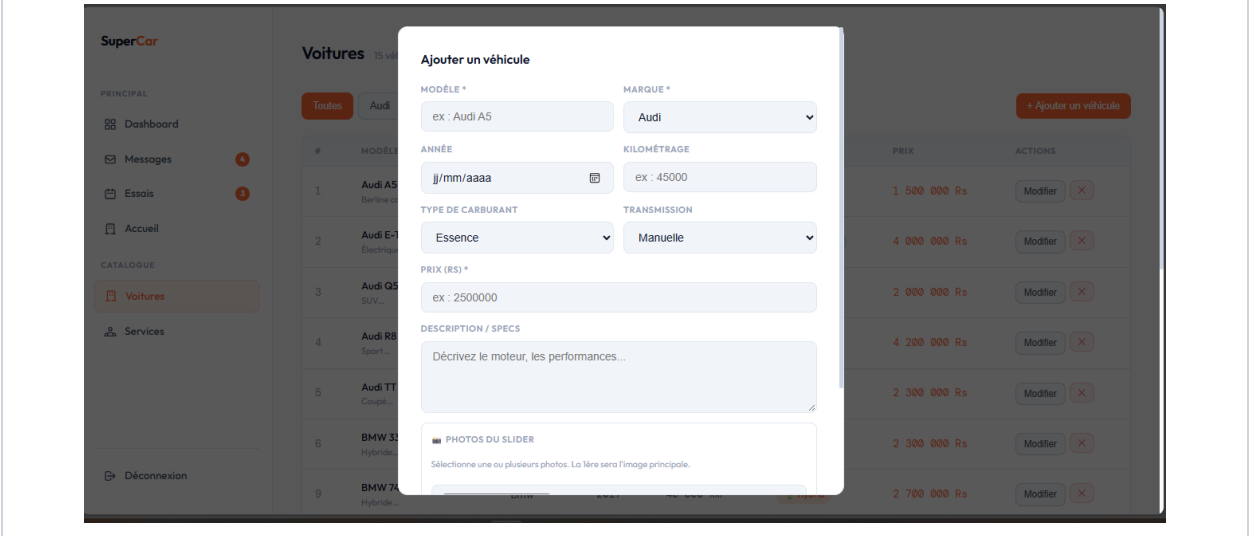


Figure 5 *– Gestion des voitures : formulaire d'ajout d'un véhicule*

12.2 Gestion des clients

La gestion des clients ne dispose pas d'une page CRUD indépendante dans le back-office : elle s'appuie sur `client_modele.php` (admin), qui permet de consulter les informations d'un client (nom, coordonnées, etc.) directement depuis la gestion des demandes d'essai et des messages de contact, chacune de ces entités étant liée à un `id_client`.

12.3 Gestion des demandes d'essai

Le module de gestion des demandes d'essai constitue l'une des fonctionnalités centrales de la partie administration du site SuperCar. Il permet aux administrateurs de consulter, traiter et suivre l'ensemble des demandes d'essai transmises par les clients depuis la partie publique du site.

Concrètement, cette page affiche la liste de toutes les demandes enregistrées en base de données, avec pour chacune les informations saisies par le client (identité, coordonnées, voiture souhaitée, date et créneau demandés) ainsi que son statut actuel (en attente, validée ou refusée). L'administrateur peut modifier ce statut directement depuis l'interface, ce qui met à jour l'enregistrement correspondant dans la base de données, et peut également supprimer une demande devenue obsolète.

Cette fonctionnalité a été développée en respectant l'architecture MVC mise en place lors de la refactorisation du projet : le contrôleur récupère les données via des requêtes préparées (PDO), et la vue se charge uniquement de leur affichage, sans logique métier mêlée au HTML.

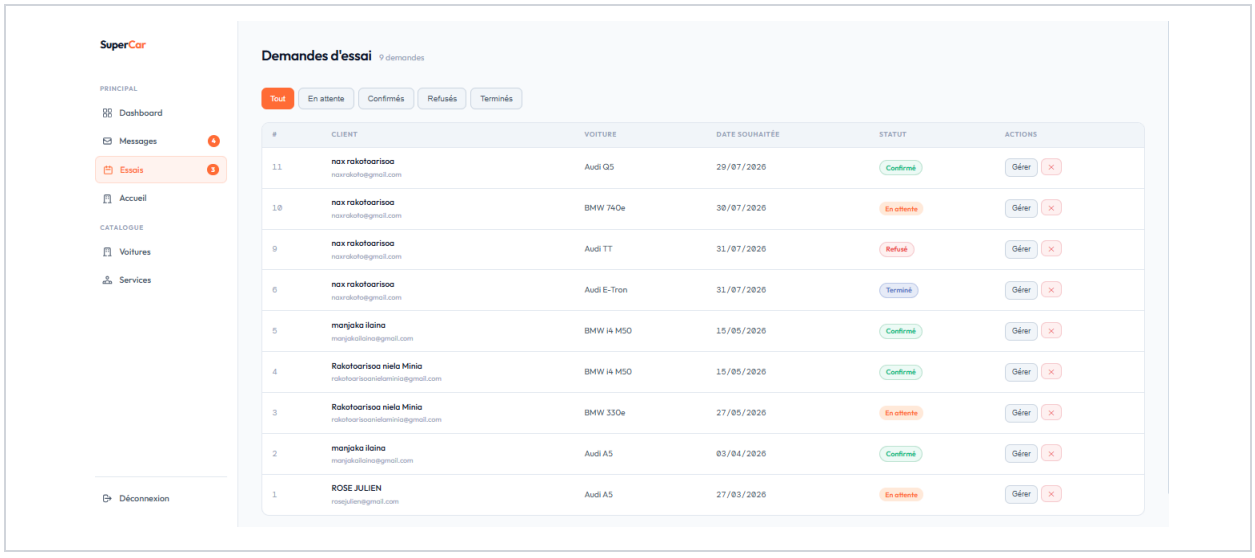


Figure 6 *– Gestion des demandes d'essai*

12.4 Gestion des messages de contact

Le module de gestion des messages de contact permet aux administrateurs de consulter les messages envoyés par les visiteurs via le formulaire de contact présent sur la partie publique

du site. Chaque message enregistré en base de données comprend les coordonnées de l'expéditeur ainsi que le contenu de sa demande.

L'interface d'administration liste ces messages par ordre chronologique, permet d'en afficher le détail complet et d'en supprimer un une fois qu'il a été traité par l'équipe commerciale. Comme pour les demandes d'essai, l'accès à cette page est réservé aux administrateurs authentifiés, et les échanges avec la base de données passent par des requêtes préparées afin d'éviter tout risque d'injection SQL.

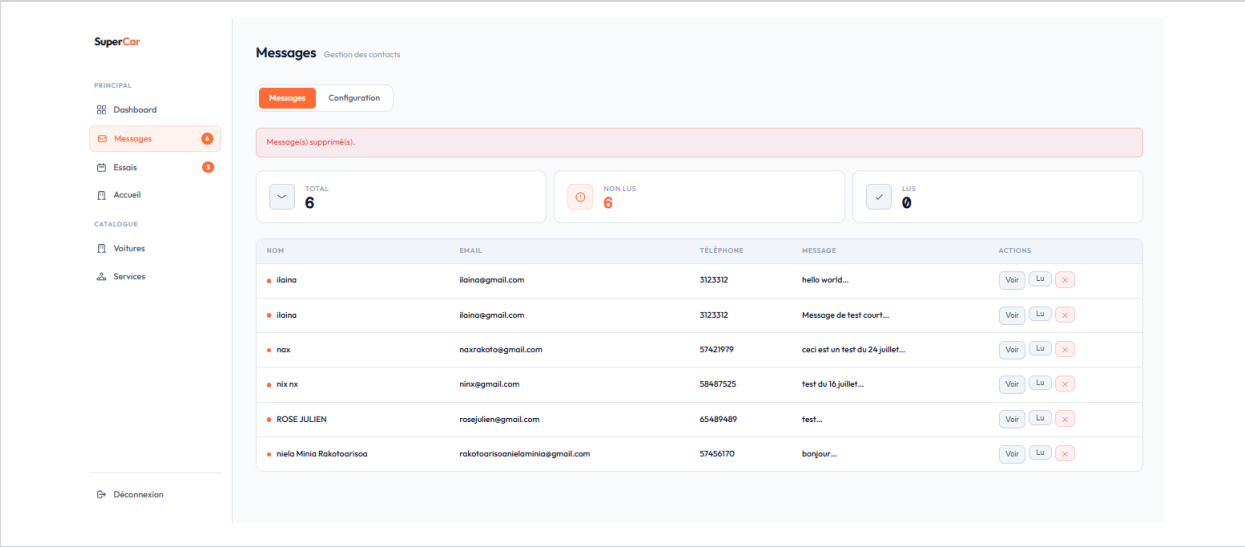


Figure 7 *– Gestion des messages de contact*

12.5 Authentification et sécurité

L'accès à l'ensemble du module d'administration est protégé par un système d'authentification. Une page de connexion dédiée demande à l'administrateur de saisir ses identifiants, vérifiés en base de données après hachage du mot de passe. Une fois connecté, une session PHP est ouverte et maintenue tout au long de la navigation dans l'espace d'administration.

Chaque page de l'administration vérifie, avant tout affichage, qu'une session valide est active ; en l'absence de session, l'utilisateur est automatiquement redirigé vers la page de connexion. Ce contrôle a été centralisé lors de la refactorisation du projet vers une architecture MVC sécurisée, afin d'éviter de dupliquer la vérification sur chaque page et de garantir qu'aucune route d'administration ne reste accessible sans authentification. Un bouton de déconnexion permet enfin de détruire proprement la session en cours.

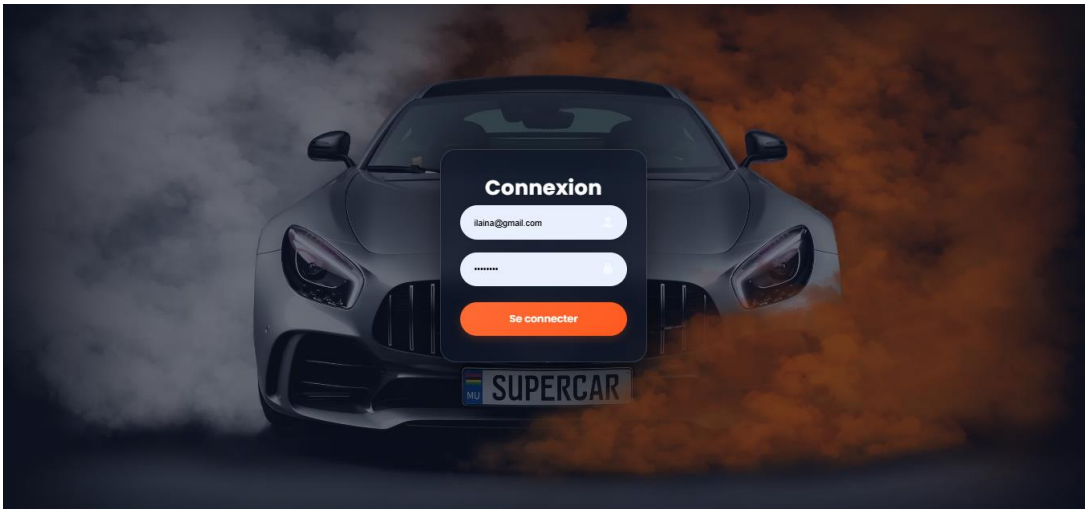


Figure 8 *– Page de connexion de l'administration*

13. Fonctionnalités spécifiques

13.1 Sélecteur de carte (Leaflet.js)

Afin d'enrichir l'expérience utilisateur, le projet intègre la bibliothèque JavaScript Leaflet.js, couplée à des fonds de carte OpenStreetMap, pour proposer un sélecteur de carte interactif. Cet outil permet de visualiser et de sélectionner une localisation directement sur une carte, sans dépendre d'un service cartographique payant.

L'intégration de Leaflet.js a nécessité de charger la bibliothèque et sa feuille de style, d'initialiser la carte sur des coordonnées par défaut, puis de gérer les événements de clic ou de sélection pour récupérer la latitude et la longitude choisies et les transmettre au formulaire correspondant. Ce composant illustre la capacité du projet à intégrer des bibliothèques tierces pour répondre à des besoins fonctionnels précis, au-delà des seules interactions PHP/MySQL classiques.

13.2 Navigation latérale partagée (sidebar)

Toutes les pages de l'administration partagent une même barre de navigation latérale (sidebar), regroupant les liens vers les différentes sections : gestion des voitures, gestion des clients, demandes d'essai, messages de contact, ainsi que le bouton de déconnexion. Le lien correspondant à la page actuellement affichée est mis en évidence, ce qui facilite le repérage de l'administrateur dans l'interface.

Sur le plan technique, cette sidebar a été isolée dans un fichier PHP unique (inclus via `include` ou `require` dans chaque page d'administration), dans la continuité du travail de modularisation et de refactorisation MVC mené sur le projet. Cette approche évite la duplication de code HTML entre les pages, simplifie la maintenance — toute modification de

la navigation ne se fait qu'à un seul endroit — et garantit une cohérence visuelle sur l'ensemble du module d'administration.

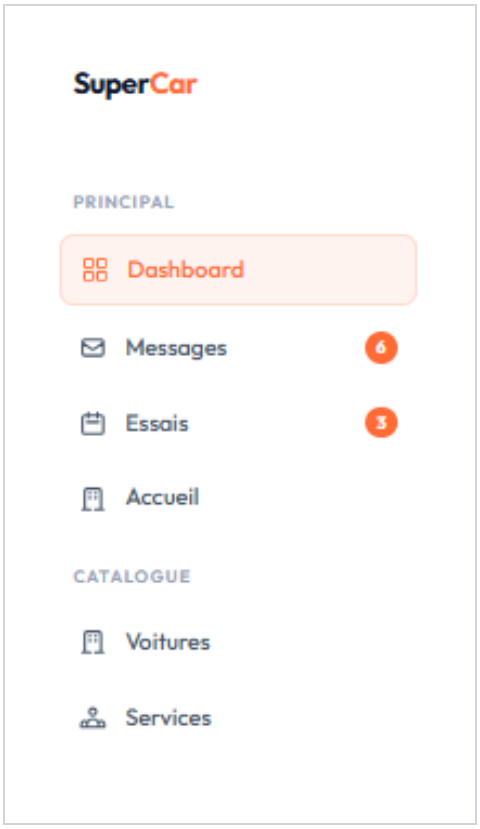


Figure 9 *– Barre de navigation latérale (sidebar) de l'administration*

14. Charte graphique et identité visuelle (thème blanc/orange)

L'identité visuelle retenue pour l'interface d'administration repose principalement sur un thème blanc associé à des touches d'orange. Le fond blanc permet d'obtenir une interface claire, lisible et agréable à utiliser, tandis que la couleur orange est utilisée pour les boutons d'action, les liens actifs et certains éléments importants afin de guider l'utilisateur vers les principales fonctionnalités.

La mise en place de cette charte graphique a représenté un travail progressif. Plusieurs feuilles de style CSS avaient été créées au fur et à mesure du développement, ce qui a entraîné certaines incohérences au niveau des couleurs, des espacements, de la typographie et de la mise en forme des différents éléments. Un travail de réorganisation et d'harmonisation a donc été commencé afin de regrouper les styles, de limiter les redondances et d'obtenir une apparence plus cohérente sur l'ensemble des pages d'administration. Cependant, certaines incohérences restent encore présentes et devront être corrigées lors des prochaines étapes du développement.

15. Difficultés rencontrées et solutions apportées

15.1 Refactorisation vers une architecture MVC

La transition vers une architecture Modèle-Vue-Contrôleur (MVC) a été une étape technique exigeante mais indispensable. L'objectif principal était de séparer l'affichage, la gestion des données et la logique de l'application afin d'obtenir un système sécurisé. Cette réorganisation a permis de rendre notre code beaucoup plus lisible, maintenable et d'éviter les failles de sécurité dans le panneau d'administration.

15.2 Consolidation et nettoyage des fichiers CSS

Au cours du développement, l'accumulation de code a créé des conflits visuels. Nous avons donc procédé à un nettoyage et à une consolidation de nos fichiers CSS. Cette étape a permis de supprimer les styles inutiles et d'unifier l'identité visuelle de l'administration autour de notre charte graphique basée sur un thème blanc et orange. Le code est désormais plus léger et l'affichage plus rapide.

15.3 Résolution par recherches en ligne et tests

Face aux différentes erreurs rencontrées, notamment en PHP ou lors de la modification des données dans l'administration, nous avons adopté une méthode de résolution systématique. Nous nous sommes appuyés sur la lecture des messages d'erreur, la consultation des documentations officielles et la recherche sur des forums spécialisés de développeurs. Chaque solution trouvée a été validée par des tests progressifs afin de s'assurer de la stabilité du code.

IV – Résultats obtenus

16. Présentation de l'administration finale

Le panneau d'administration final est aujourd'hui pleinement fonctionnel et sécurisé. Il offre une interface centralisée et intuitive permettant aux gérants de la concession de piloter leur activité. Depuis ce tableau de bord, l'administrateur peut facilement gérer le catalogue des voitures (ajout, modification, suppression), consulter la base de données des clients, et traiter efficacement les formulaires de contact ainsi que les demandes d'essai.

17. Tests et validation

Avant le déploiement définitif, le projet a été soumis à une phase de tests rigoureux. Nous avons mené des tests fonctionnels pour garantir que toutes les opérations (formulaires, interactions avec la base de données) s'exécutent sans erreurs. De plus, des tests de compatibilité ont été réalisés pour s'assurer que le design responsive de l'administration s'adapte parfaitement aux différents supports, qu'il s'agisse d'ordinateurs, de tablettes ou de smartphones.

18. Analyse de la conformité aux besoins exprimés

L'outil final correspond parfaitement au cahier des charges et aux attentes de l'organisation cliente. Le panneau d'administration modernise la gestion interne en automatisant et en centralisant le traitement des données.

19.Hebergement

1. Hébergement et transfert

Le site est hébergé chez Alwaysdata. Les fichiers sont envoyés avec FileZilla en SFTP, un protocole qui chiffre les échanges, contrairement au FTP classique. La connexion se fait avec l'hôte ssh-nix.alwaysdata.net, mon identifiant de compte et le port 22 par défaut. À la connexion, FileZilla affiche le contenu du dossier /home/nix, la racine de mon compte.

2. Organisation des fichiers sur le serveur

Le site est configuré pour pointer directement vers le dossier supercar. Seul ce dossier est accessible depuis le web. Tout le reste est hors de la racine web.

Emplacement	Contenu	Accessible par URL
/home/nix/supercar	Pages PHP publiques (connexion.php, contact.php, evenement.php…), dossiers css, js, Images-Index, ADMIN, fichier .htaccess	Oui
/home/nix/config, controleurs, modeles, vues, includes	Configuration et code MVC	Non

3. Intérêt de cette architecture

- Sécurité : les fichiers de configuration, le code métier ne peut pas être appelé par un visiteur, même en cas de dysfonctionnement de PHP.
- Respect du MVC : les pages de supercar servent de points d'entrée et appellent les contrôleurs situés dans le dossier parent.
- Maintenance : la logique et les ressources publiques se mettent à jour séparément.

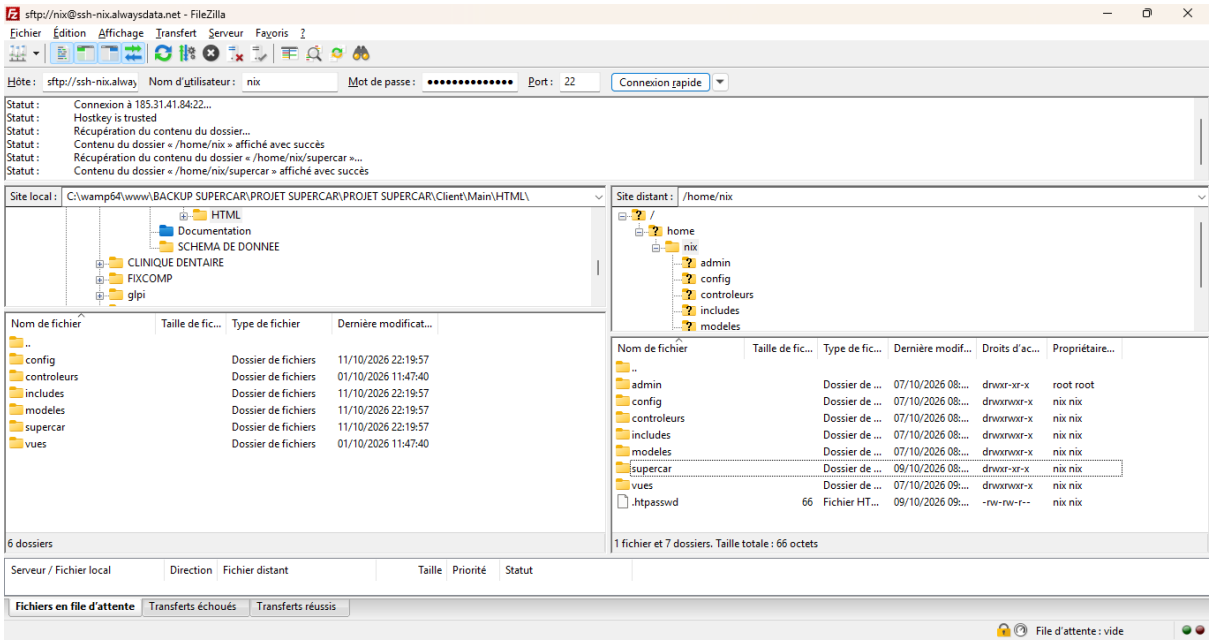
4. Procédure de déploiement

1. Se connecter en SFTP avec FileZilla.

2. Côté local, ouvrir le dossier HTML du projet. Côté distant, rester dans /home/nix.

3. Glisser-déposer les dossiers du projet : supercar pour la partie publique, les autres à la racine du compte.

4. Contrôler l'onglet « Transferts réussis », puis tester le site dans le navigateur.



Conclusion

Bilan technique et personnel

Le projet SuperCar nous a permis de mettre en pratique nos connaissances en développement web à travers la création d'une application dynamique. Nous avons travaillé sur la conception du site, la base de données, l'administration et les différentes fonctionnalités. Ce projet nous a également appris à mieux organiser notre travail en équipe et à utiliser Git pour gérer le développement.

Apprentissages et compétences développées

Ce projet nous a permis de développer nos compétences en HTML, CSS, PHP, MySQL, JavaScript, Bootstrap et MVC. Nous avons également appris à utiliser PDO, à sécuriser l'accès à l'administration et à gérer les données à travers une base de données. Le travail en groupe nous a aussi permis d'améliorer notre organisation et notre collaboration.

Perspectives et améliorations futures

Plusieurs améliorations peuvent encore être apportées au projet, notamment la finalisation de certains éléments graphiques, l'harmonisation des styles CSS et l'amélioration de l'interface d'administration. De nouvelles fonctionnalités pourraient également être ajoutées afin de rendre l'application plus complète et plus pratique pour les administrateurs.